BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

http://fit.vut.cz

Project practice
2025/2026

LLM-Based Translation Pipeline for Functional Coverage Analysis

Pavel Lukl*

Abstract
Safety-critical avionics software requires formal coverage analysis of requirements, but requirements are written in natural language. Manual formalization is slow, error-prone, and does not scale. This report investigates the use of large language models (LLMs) in a multi-step agentic pipeline to automatically translate natural-language requirements into a domain-specific predicate formalism. We survey existing approaches to requirement formalization, design a four-step pipeline (ambiguity review, flavour extraction, entity extraction, and constraint formalization with structural validation), and conduct preliminary experiments comparing orchestration frameworks, output representations (JSON vs Python), task decomposition strategies (multi-call pipeline vs single-call chain-of-thought), and local vs cloud models. Preliminary results suggest that model capability may be the strongest predictor of output quality, that the choice of output format interacts with model size in ways not predicted by prior work, and that structured decomposition in prompts improves results regardless of architecture. These observations require confirmation through larger-scale testing. We identify open questions including comprehensive evaluation across models and requirements, human-in-the-loop review, semantic validation, and iterative improvement through fine-tuning.

Keywords: DO-178C — LLM — requirement formalization — agentic pipeline — functional coverage

Supplementary Material: *N/A*

*xluklp00@stud.fit.vut.cz, *Faculty of Information Technology, Brno University of Technology*

1. Introduction

Safety-critical avionics software must demonstrate that tests adequately cover the behaviors specified in requirements. This coverage analysis requires a formal representation of the requirements, yet in practice, both requirements and test cases are written in natural language. Without formalization, inconsistencies between requirements and tests can go undetected, potentially compromising the safety of the final system. Translating them into a formal form is therefore essential, but it is currently a manual process: slow, expensive, and prone to human error. As the number of requirements grows, manual formalization becomes a bottleneck in the certification workflow.

The broader project addresses this by building an end-to-end system: natural language requirements and test cases are translated into a formal representation, the translations undergo thorough validation, and finally a gap analysis identifies missing coverage between the formalized requirements and test cases. This report focuses on the first stage: automating the translation using large language models (LLMs). Other parts of the project (the formal representation itself, the semantic validation of translations, and the gap analysis) are being developed in parallel by other team members. Figure 1 shows the overall project pipeline; this report addresses the LLM-Based Translation stage.

Rather than relying on a single LLM prompt, we design an agentic pipeline of several steps, where each step produces inspectable, validated output, a property essential in safety contexts where traceability of fail-

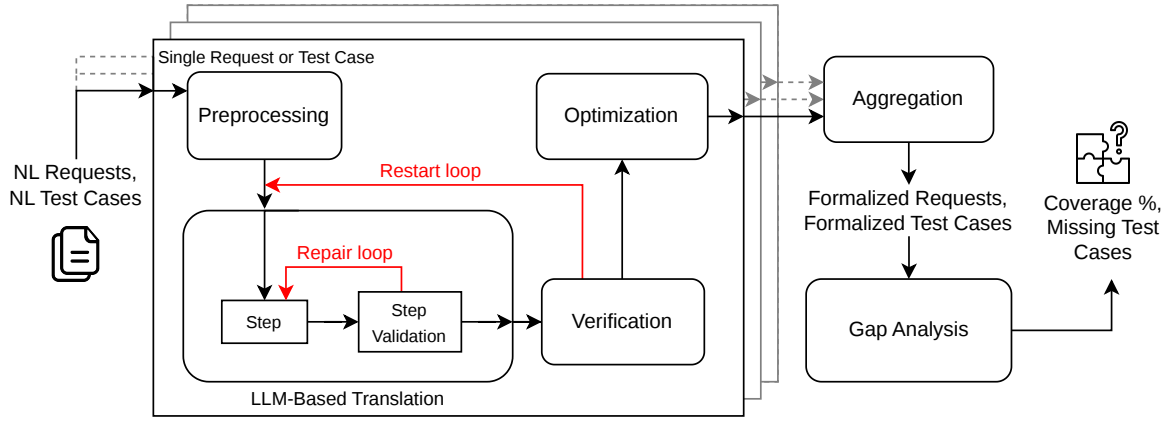

Figure 1. Overview of the full project pipeline. This report focuses on the LLM-Based Translation stage (center).

ures is required. The pipeline takes a natural language requirement as input and produces a formal predicate tree conforming to a formalism developed within the project.

We begin by surveying existing approaches to requirement formalization and the technologies available for building LLM pipelines in this context (Section 2). Drawing inspiration from these, we draft our own approach to the translation problem, describe the pipeline architecture and the design decisions behind it, and present initial experiments comparing four orchestration frameworks on real avionics requirements (Section 3). Based on the findings, we outline concrete directions for future development as the project continues (Section 4).

2. State of the Art

Translating natural language requirements into formal representations has traditionally been a manual process requiring domain expertise. Recent advances in large language models have opened the possibility of automating this translation, and a growing body of work explores different strategies. A recent systematic literature review by Beg et al. [2] identifies 35 studies on the topic published between 2022 and mid-2025. We survey the relevant approaches below, organized into four areas: requirement quality and pre-processing (Section 2.1), formalization approaches (Section 2.2), output representation and correctness (Section 2.3), and implementation technologies (Section 2.4).

2.1 Requirement Quality and Pre-processing

Before a requirement can be formalized, it must be sufficiently unambiguous and complete. Template-based tools such as FRET [9] achieve precise formalization by having users write requirements in a restricted natural language with unambiguous semantics, but require every requirement to be manually rewritten to conform to the template grammar. NLP-based defect detection offers an alternative: pattern-based approaches have been applied to identify ambiguities and inconsistencies in requirements in safety-critical domains such as railway [6]. More recently, Mahbub et al. [13] evaluated GPT-4 for automated detection of ambiguities, inconsistencies, and incompleteness in industrial requirements, finding that while LLMs show promise for early defect screening (precision 0.61 for incompleteness), they are less reliable for ambiguity detection (precision 0.39) and cannot replace human analysts. These findings suggest that LLM-based ambiguity review is useful as a pre-processing step that flags potential issues for human review, but should not autonomously rewrite requirements.

2.2 Formalization Approaches

Interactive decomposition and dynamic prompting. Cosler et al. [3] propose decomposing the formalization into sub-translations: the LLM maps fragments of the input to sub-formulas, which serve as a structured intermediate representation that users can inspect, edit, and correct before the final formula is assembled. This decomposition makes the model's reasoning transparent at each step, and the interactive loop lets users resolve ambiguities that single-pass approaches cannot reliably detect. The framework, called *nl2spec*, targets LTL and similar temporal logics. Xu et al. [15] take the prompting strategy further with dynamic few-shot prompting (where a small number of solved examples are included in the prompt to guide the model): instead of using a fixed set of examples, the system retrieves the most relevant ones from a prompt library via embedding similarity for each input, and evolves the library over time by incorporating corrected translations. This achieves up to 94.4% accuracy without

human interaction. With interactive prompt evolution, accuracy on an out-of-domain dataset improves from 27% to 78%. Both approaches demonstrate that careful prompt selection and human-in-the-loop refinement can compensate for a model's limited domain knowledge, and that inspectable intermediate outputs are valuable for catching errors early.

Structured decomposition and agentic pipelines. Khot et al. [11] show that decomposing complex tasks into sub-tasks delegated to specialized prompts outperforms single-pass chain-of-thought, since each prompt can be independently optimized for its sub-task. Several recent works apply this principle to requirement formalization, separating semantic reasoning from formal output generation using structured intermediate representations. Ma et al. [12] propose REQ2LTL, where the LLM performs semantic reasoning about the requirement and produces output in a hierarchical intermediate representation called OnionL, a tree of scopes, relations, and atomic propositions. A deterministic rule-based engine then validates this tree and translates it to LTL. The LLM is called with a single prompt containing the grammar of the intermediate language, a step-by-step decomposition algorithm as chain-of-thought instructions, several ground-truth examples of requirement-to-tree translations, and the requirement itself. This achieves 100% syntactic correctness and 88.4% semantic accuracy on 112 real aerospace requirements, significantly outperforming other approaches. The framework also supports optional human validation via automatically generated visual tree diagrams. An ablation study confirms that removing either the intermediate representation or the chain-of-thought decomposition degrades accuracy by over 20 percentage points. Yan et al. [16] apply a similar separation in hardware verification: the first LLM converts unstructured text into structured descriptions using a unified template, the second generates assertions from these descriptions, and the third validates the results. Nouri et al. [14] use the same principle in the automotive safety domain, breaking the analysis into smaller tasks with a dedicated prompt for each, where each step's output feeds the next without modification. Their key finding is that this decomposition significantly outperforms a single monolithic prompt. Together, these works establish that separating semantic extraction from formal translation, with validated intermediate representations between stages, is an effective strategy for complex formalization tasks.

Fine-tuning. Fine-tuning language models on datasets of natural language sentences paired with their formal translations [10] can achieve competitive accuracy. A key advantage is that fine-tuned models generalize to unseen variable names, a property that rule-based approaches lack. However, fine-tuning requires a sufficiently large training dataset, which does not exist for specialized domains such as avionics. In practice, such a dataset could be accumulated incrementally: a prompt-based pipeline with human-in-the-loop review produces corrected formalizations that serve as training pairs over time, enabling fine-tuning as a downstream improvement rather than a prerequisite.

2.3 Output Representation and Correctness

A recurring challenge across all formalization approaches is ensuring that the generated output is not only structurally valid but also semantically correct. This subsection surveys strategies for improving output correctness, from constraining the generation process to choosing the right output format.

Constrained decoding. Grammar-Forced Translation (GraFT) by English et al. [5] constrains the language model's token generation at each step so that only tokens permitted by the target grammar can be produced. This provides a formal guarantee that every generated formula is syntactically valid, eliminating the need for structural retries entirely. The technique involves two steps transferable to other formalisms: first, a masked language model lifts atomic propositions from the natural language input (replacing domain terms with integer placeholders), then a fine-tuned sequence-to-sequence model generates the formula under grammar constraints. In their evaluation on temporal logic benchmarks, this improves end-to-end accuracy by 5.49% and out-of-distribution accuracy by 14.06% over prior methods.

Code as output representation. The idea of using code rather than custom notation as the output format has broader support. Gao et al. [7] show that having LLMs generate Python programs instead of reasoning directly, then executing the code, outperforms chain-of-thought on mathematical and symbolic reasoning tasks by offloading the solution step to an interpreter. Danso et al. [4] provide evidence specific to formalization: in a systematic evaluation of several LLMs on NL-to-LTL translation, they compare three prompt formats with increasing structural constraint (minimal, detailed grammar, and a Python interface where each temporal operator is a Python dataclass constructor). Reformulating the task as Python code generation improves semantic equivalence from approximately 24% to over 70%, and raises syntactic correctness to near 100%. The improvement is attributed to two factors:

LLMs have far more Python training data than LTL notation, and the dataclass constructors physically prevent certain classes of malformed output.

The syntax–semantics gap. Critically, Danso et al. show that the dominant source of error is not syntax but semantics. While models produce syntactically valid formulas most of the time, they frequently get the *meaning* wrong. The most common errors are: confusion between the "always" (G) and "eventually" (F) operators, incorrect nesting of temporal scopes (placing an operator at the wrong level of the formula tree), and over- or under-specification where the generated formula is strictly stronger or weaker than intended. Atomic proposition extraction is relatively reliable (57.2% overlap), but operator agreement drops to 36.7% and exact structural alignment to 18.7%. These semantic errors are invisible to structural validation and can only be detected through formal equivalence checking, which the authors perform using the NuSMV model checker as an external oracle. This finding is echoed by REQ2LTL [12], whose ablation study shows that structured decomposition addresses precisely these semantic errors, improving accuracy by over 20 percentage points. LLM compliance with complex JSON schemas also degrades sharply as schema depth increases [8], motivating either simpler per-step schemas or alternative output representations.

2.4 Technologies

Building a formalization pipeline requires concrete choices about how the steps are orchestrated and how the LLM is accessed. Several open-source frameworks have emerged for this purpose, and a recent comparison of ten such frameworks on the same task found significant differences in implementation complexity and reliability [1]. PydanticAI provides the most direct integration with Pydantic output schemas, with automatic retry on validation failure built in. Haystack models pipelines as directed acyclic graphs of typed components, with native support for serialization to YAML. LangGraph represents pipelines as state graphs with conditional edges and supports cycles, which naturally accommodates loops that retry on validation failure. CrewAI takes a role-based approach, defining agents with goals and backstories that collaborate on tasks. Other notable frameworks include DSPy, which optimizes prompts programmatically, and Instructor, which wraps function-calling APIs with Pydantic validation. The choice of framework determines how easily the pipeline can be inspected at intermediate steps, how schema changes propagate, and how validation errors are surfaced, all of which directly affect auditability. The LLM itself can be accessed either through cloud APIs (such as Claude or GPT-4) or by running open-weight models locally using tools like Ollama. Most orchestration frameworks support both options, allowing the same pipeline to switch between them with minimal code changes.

Summary

The surveyed literature shows that LLMs can translate natural language requirements into formal representations with increasing accuracy, and that the most effective approaches share several methodological patterns: task decomposition into smaller, individually verifiable steps; structured intermediate representations between stages; feedback-driven refinement through human interaction, dynamic prompt evolution, or automated retry; and careful choice of output representation, with Python code outperforming custom notation on frontier models [4, 7]. A recurring finding is that syntactic correctness is largely solved, while the real challenge lies in semantic correctness, particularly mis-scoped temporal operators and incorrect nesting [4, 12]. The closest existing work is REQ2LTL [12], which targets aerospace requirements and uses a hierarchical intermediate representation conceptually similar to our formalism. However, their output is standard LTL and their translation step is a fixed rule-based engine, whereas our formalism is domain-specific and still evolving, requiring the translation step itself to use LLM generation with schema validation. Our work builds on the same principles demonstrated by REQ2LTL and the other surveyed approaches, applied to this different setting.

3. Our Approach

3.1 Research Questions

Based on the patterns and open questions identified in the literature, we investigate four research questions:

- **RQ1: How should the formalization task be decomposed?** The literature shows that both single-call chain-of-thought decomposition [12] and multi-call pipelines [14, 11] outperform monolithic prompts, but does not establish which is preferable for a given formalism and model capability level.
- **RQ2: How does output representation affect formalization accuracy?** Danso et al. [4] and Gao et al. [7] report that Python code output significantly outperforms custom notation on frontier models. We test whether this finding holds for smaller local models and for our specific formalism.

- **RQ3: How does model capability affect architecture choices?** Cloud frontier models and local open-weight models differ substantially in reasoning ability. We investigate whether the optimal pipeline architecture depends on the model used.
- **RQ4: Which orchestration framework best fits this use case?** Ahmed et al. [1] show significant differences in framework complexity and reliability. We compare four frameworks on the same formalization task.

3.2 The Formalism

The target representation is a domain-specific predicate formalism designed for expressing avionics software requirements and test cases in a form suitable for mechanical coverage analysis. We summarize the aspects relevant to the pipeline.

A formalized requirement is a triple R consisting of a *flavour* (Discrete or Continuous, fixing the time model), a set of *entities*, and a *constraint* (a temporal construct expressing the required behavior). Entity types include Signal, Storage, Event, EventTrigger, Channel, State, Value, Abstract, and Set, each with a defined set of permitted operations (e.g., `fire` for EventTrigger, `write` for Storage). Temporal constructs such as Always, Eventually, Causes, CausesWithin, Sequence, and others quantify over time, and predicates within them reference entity values, event occurrences, and interval-based computations.

The formalism evolves between versions as new entity types and operations are added, but each individual translation run targets a fixed schema version. This means the pipeline does not need to handle schema changes at runtime; instead, the schema is updated between versions and the pipeline (including any fine-tuned models or rule-based components) is updated accordingly.

The formalism can be mapped to LTL and similar temporal logics, which makes the LTL-focused literature from Section 2 directly informative. Our pipeline targets the formalism itself as its output, analogous to how REQ2LTL [12] targets OnionL as its intermediate representation. Once the formalism stabilizes and a mapping to temporal logics is defined, a deterministic rule-based translation to LTL could be added as a downstream step, mirroring REQ2LTL's architecture.

3.3 Pipeline Architecture

Following the decomposition principle established in the literature [11, 14], our pipeline breaks the formalization into four steps, each producing inspectable, validated output (Figure 2):

1. **Ambiguity Review.** The LLM flags potential ambiguities in the requirement text against a checklist (temporal ambiguity, scope of negation, missing causes, vague language). Following Mahbub et al.'s [13] finding that LLMs are unreliable for autonomous ambiguity resolution, this step flags issues for human review but does not rewrite the requirement. The ambiguity notes are passed as context to all subsequent steps.
2. **Flavour Extraction.** The LLM determines whether the requirement uses discrete time (step-based behavior, register operations) or continuous time (real-time durations, deadlines). This is a simple classification that sets the time model for the rest of the pipeline.
3. **Entity Extraction.** The LLM identifies all entities referenced in the requirement, producing a structured intermediate representation: each entity is a tuple of (name, type, modifiers, description). This step serves the same architectural role as the intermediate representation in REQ2LTL [12] and AssertLLM [16]: it separates semantic extraction from formal output generation and provides an inspection point where entity types and modifiers can be validated before the constraint is built.
4. **Constraint Formalization.** Given the entities, flavour, and ambiguity notes, the LLM constructs the temporal constraint. The prompt includes a step-by-step decomposition algorithm (inspired by REQ2LTL's BUILDING-ONIONL [12]): identify the top-level temporal scope, decompose the condition into sub-predicates, decompose the effect, and normalize entity references. If structural validation fails, the error is fed back and the step retries automatically.

Each step uses a dedicated system prompt consisting of three parts: a role and goal statement, the formalism context (entity types, temporal constructs, predicates, and expressions with their schemas), and step-specific instructions with worked examples. The formalism context is shared across steps so that the LLM has the same reference material regardless of which step it is executing. The constraint formalization step additionally includes a decomposition algorithm as chain-of-thought instructions, following the pattern established by REQ2LTL [12]. Ambiguity notes from step 1 are appended to the prompts of all subsequent steps as additional context. The full prompts are included in Appendix A.

Structural validation checks that every entity ref-

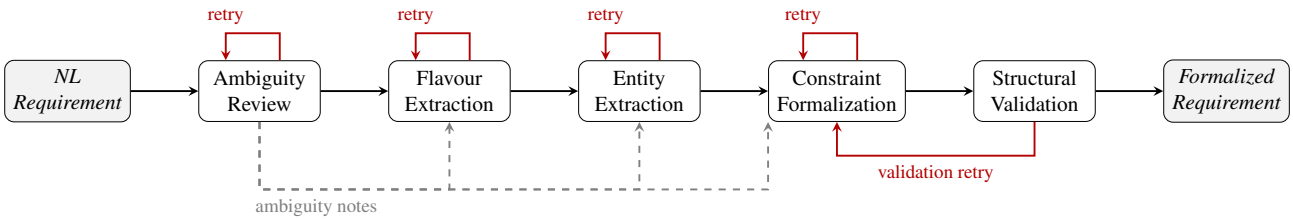

Figure 2. Detailed view of the LLM-Based Translation stage from Figure 1. The Preprocessing block from the overview corresponds to our Ambiguity Review and Flavour Extraction steps; the Step and Step Validation blocks correspond to Entity Extraction and Constraint Formalization with their respective validations. Each LLM step automatically retries on schema validation failure (red self-loops). Structural validation additionally triggers a retry of the constraint formalization step (red arrow from validation). Ambiguity notes from the first step are passed as context to all subsequent steps (dashed arrows).

erenced in the constraint was declared in step 3, that operations are compatible with entity types, and that the output conforms to the Pydantic schema. This catches structural errors but not semantic ones: as Danso et al. [4] show, a formula can be structurally valid while meaning something entirely different from the requirement. Semantic validation is a separate concern outside the scope of this work.

3.4 Preliminary Experiments

We conducted four sets of preliminary experiments to inform the research questions. Due to the exploratory nature of this phase, sample sizes are small and results should be interpreted as directional findings, not definitive answers. Each experiment's setup (model, schema version, requirements tested) is noted explicitly, as these varied across experiments.

RQ4: Framework comparison. We implemented the pipeline in four orchestration frameworks (PydanticAI, Haystack, LangGraph, CrewAI) and ran 48 tests (4 frameworks $\times$ 4 requirements $\times$ 3 runs) using a locally served qwen2.5:14b model and an earlier, simpler version of the formalism schema. Three frameworks achieved 92% structural success rate; LangGraph reached 75%. CrewAI was consistently fastest (25.5s average vs 50–66s for others) and produced the deepest formula trees. Investigation via HTTP request interception revealed that CrewAI omits the `response_format: json_object` parameter that the other frameworks send, but reproducing this change in PydanticAI did not reliably replicate the advantage. PydanticAI required the least code ($\sim$140 lines vs $\sim$200–220) and offered the best schema propagation: changing the Pydantic model automatically updates the LLM's schema context. Table 1 summarizes the results.

	PydanticAI	Haystack	LangGraph	CrewAI
Success rate	92%	92%	75%	92%
Avg time (s)	65.0	65.6	50.5	**25.5**
Avg attempts	1.4	1.8	1.8	**1.3**
Lines of code	$\sim$**140**	$\sim$205	$\sim$210	$\sim$220

Table 1. Framework comparison (qwen2.5:14b, 4 requirements, 3 runs each). Success means the output is structurally valid against the schema.

RQ2: Output representation. To test Danso et al.'s [4] finding that Python output improves formalization accuracy, we ran 10 requirements $\times$ 3 runs in both JSON and Python output modes using phi4:14b (a local model) and the updated formalism schema. The LLM received the same formalism context in both cases; only the output format instructions differed. JSON achieved 90% structural success (27/30) while Python achieved 70% (21/30). Python errors included inventing function names, wrong argument counts, and confusing API levels. JSON was also faster (17.3s vs 20.2s average). This contradicts Danso et al.'s finding, but their evaluation used frontier models (GPT-4o, Claude, Gemini). Our result suggests that the Python advantage is capability-dependent: it emerges when the model is fluent enough in Python to benefit from the structural scaffolding, but a smaller model makes more errors with an unfamiliar API than with JSON discriminators it has seen in training. Table 2 shows the per-requirement breakdown.

Req	JSON	Python	Req	JSON	Python
01	1/3	2/3	06	3/3	3/3
02	3/3	3/3	07	3/3	3/3
03	3/3	3/3	08	3/3	3/3
04	3/3	2/3	09	3/3	0/3
05	3/3	1/3	10	2/3	1/3
Total				**27/30**	**21/30**

Table 2. Python vs JSON structural success rate (phi4:14b, 3 runs per requirement). Success means the output validates against the Pydantic schema.

RQ1: Task decomposition. We compared two architectures: the multi-call pipeline (separate LLM calls for flavour, entities, and constraint with validation between steps) and a single-call chain-of-thought approach (one prompt containing a step-by-step decomposition algorithm, inspired by REQ2LTL [12]). Both used identical decomposition logic; only the execution model differed. On phi4:14b, the pipeline achieved 75% success vs 50% for CoT (4 requirements, 1 run). On Gemini 3.5 Flash (a cloud model), both achieved 100% (4 requirements, 1 run). Ground truth comparison revealed that CoT produced better temporal precision on complex requirements (correctly using `LastOcc` to reference values at past event times), while the pipeline produced more thorough entity extraction. An informal test with a frontier model (Claude) on the hardest requirement produced the closest match to ground truth of any configuration, correctly capturing temporal lookups that all other setups missed. These results suggest that weaker models benefit from the pipeline's simpler per-call schemas, while stronger models benefit from CoT's full-context reasoning. Table 3 summarizes the results.

	phi4:14b		**Gemini 3.5 Flash**	
Req	Pipeline	CoT	Pipeline	CoT
02	✓	✓	✓	✓
03	✓	×	✓	✓
06	✓	✓	✓	✓
09	×	×	✓	✓
Total	**3/4**	**2/4**	**4/4**	**4/4**

Table 3. CoT vs Pipeline structural success rate (4 requirements, 1 run each). Checkmark indicates the output validates against the schema. Gemini results use the updated decomposition prompt.

RQ3: Model capability. Across all experiments, model capability was the strongest predictor of output quality. The local phi4:14b model achieved 75–90% structural success on simple to medium requirements but failed consistently on the most complex requirement (Req 9: multi-condition timed toggle with event counting and interval construction). Gemini 2.5 Flash and 3.5 Flash succeeded on this requirement in both pipeline and CoT modes. The semantic quality gap was even larger: cloud models correctly used advanced constructs such as event occurrence counting with open intervals and value lookups at past event times, which local models either simplified away or produced incorrectly. Reasoning-specific model variants were incompatible with framework-managed structured output due to embedded thinking tags that interfere with JSON parsing.

3.5 Discussion

The preliminary experiments confirm several findings from the literature and reveal one notable divergence. Task decomposition improves formalization quality regardless of whether it is implemented as a multi-call pipeline or as chain-of-thought instructions within a single call, confirming the core finding of Khot et al. [11] and REQ2LTL [12]. The choice between the two architectures depends on model capability: smaller models need the simpler per-step schemas that a pipeline provides, while larger models benefit from seeing the full requirement context in a single call.

The finding that JSON outperforms Python on a 14B local model (RQ2) is the clearest divergence from the literature. Danso et al. [4] measured semantic equivalence improvements of nearly threefold with Python output, but their evaluation used frontier models. Our result suggests that the advantage is not inherent to the output format but depends on the model's fluency with the representation: a model that has seen far more JSON schemas in training than custom Python APIs will perform better with JSON. This has practical implications for deployment in environments where cloud models are not available.

All findings reported above are based on small sample sizes (1–3 runs per configuration, 4–10 requirements) and should be treated as preliminary observations rather than confirmed results. Confirming these trends requires more comprehensive testing: larger requirement sets, multiple runs per configuration for statistical significance, a wider range of models, and systematic ground-truth evaluation. The resources required for such testing, particularly access to frontier cloud models with sufficient API quotas, were not available during this phase of the project.

Beyond these experimental questions, several design decisions remain open, including the degree of human involvement, the choice of semantic validation strategy, and the path toward fine-tuning. We discuss these in Section 4.

4. Open Questions and Future Directions

The preliminary findings from this phase raise several questions that the next phase of the project will need to address through more comprehensive experimentation.

Comprehensive evaluation. The experiments reported in this work used small sample sizes and a limited set of models. Confirming the observed trends requires testing on the full set of avionics requirements with multiple runs per configuration, a broader range of models (including frontier cloud models with sufficient API access), and systematic comparison against manually produced ground-truth formalizations. Statistical significance testing is needed before any of the current findings can be stated as definitive results.

Pipeline granularity. Our preliminary results suggest that model capability determines whether a single-call or multi-call architecture is preferable, but this was observed on only four requirements with one run each. A controlled experiment varying model size, requirement complexity, and prompt structure is needed to establish when to use chain-of-thought within a single call and when to decompose into separate pipeline steps. The current pipeline's constraint formalization step may itself benefit from further decomposition, for example splitting temporal scope identification, condition construction, and effect construction into separate calls with validation between them. The optimal level of decomposition is an open question that likely depends on both the model and the complexity of the requirement.

Output representation. The finding that JSON outperforms Python on smaller models while the literature reports the opposite for frontier models needs further investigation. Testing with additional output formats, fine-tuned models familiar with the formalism's Python API, and a wider range of model sizes would clarify whether the advantage is a function of model capability, API familiarity, or schema complexity. Rather than committing to a single output format, a promising approach is to generate both JSON and Python output in parallel and select the result that passes validation, or scores higher when both are structurally valid. This ensemble strategy avoids the need to choose a single format and exploits the fact that the two representations tend to fail on different inputs.

Semantic validation. While semantic validation is outside the scope of this work, it is important to outline the problem since it directly affects the trustworthiness of the pipeline's output. Structural validation alone cannot detect the dominant class of errors: semantically incorrect but syntactically valid formulas. Possible approaches include roundtrip verification (translating the formula back to natural language and comparing with the original requirement), formal equivalence checking via model checkers once the formalism-to-LTL mapping is available, and human expert review. Another possible approach is adversarial LLM verification: after the pipeline generates output (a positive call), a second LLM call is prompted to challenge it (a negative call), asking targeted questions such as whether a specific entity from the requirement was missed, whether the temporal scope is correct, or whether the effect fully captures the intended behavior. The generator and the critic have different biases, so combining them can catch errors that neither would detect alone. This is the most critical open problem for making the pipeline trustworthy enough for use in safety-critical contexts.

Dynamic example selection. Following Xu et al. [15], retrieving the most relevant few-shot examples for each input via embedding similarity could improve accuracy, particularly as the library of correctly formalized requirements grows over time. This has not yet been tested.

Deterministic translation to temporal logics. Once the formalism stabilizes, a rule-based translator from the formalism to LTL (analogous to REQ2LTL's OnionL-to-LTL engine [12]) would enable formal verification of the output using established model checkers. This would also provide a path to semantic validation through equivalence checking.

Human-in-the-loop design. The pipeline will likely incorporate human review at key stages, as in nl2spec [3] and REQ2LTL [12]. In a safety-critical context, autonomous formalization without expert oversight is unlikely to be acceptable. The specific review points (after entity extraction, after constraint generation, or both) and the interface for presenting intermediate results to reviewers remain to be designed.

Fine-tuning as iterative improvement. While fine-tuning was ruled out for the initial implementation due to the lack of training data [10], it becomes a viable improvement path once human-in-the-loop review is in place. Each human-corrected formalization produces a validated requirement-to-formalism pair that can be accumulated into a training dataset over time. A model fine-tuned on these corrections would gradually reduce the rate of errors and the amount of human intervention needed, creating a feedback loop between human review and model improvement.

5. Conclusion

This report investigated the use of large language models for automating the translation of natural-language avionics requirements into a domain-specific predi-

cate formalism. We surveyed existing approaches to requirement formalization, identified key methodological patterns in the literature, designed a four-step pipeline with structural validation and retry, and conducted preliminary experiments comparing orchestration frameworks, output representations, task decomposition strategies, and model capability levels.

Four findings emerged from the experiments. First, model capability is the strongest predictor of output quality: a cloud model (Gemini) succeeded on all tested requirements including the most complex one, while a local 14B model failed consistently on complex requirements regardless of framework or prompt design. Second, JSON output outperformed Python on the local model (90% vs 70% structural success), contradicting prior work on frontier models and suggesting that the output format advantage depends on model capability. Third, chain-of-thought decomposition within a single call produced better temporal precision than a multi-call pipeline on stronger models, but weaker models benefited from the pipeline's simpler per-step schemas. Fourth, structured decomposition in the prompt improved results regardless of architecture, confirming findings from the literature.

These results are preliminary and based on small sample sizes. The next phase of the project will focus on comprehensive evaluation with larger requirement sets and multiple models, integration of human-in-the-loop review, and iterative improvement through fine-tuning on corrected formalizations. Semantic validation, the most critical open problem, remains outside the scope of the translation pipeline and will be addressed separately.

References

[1] AHMED, S. R. and WEI, W. DDL2PropBank Agent: Benchmarking Multi-Agent Frameworks' Developer Experience Through a Novel Relational Schema Mapping Task. *ArXiv preprint*. 2026.

[2] BEG, A., GERVASI, V., VIDANAGE, K. and LIYANAGE, L. H. A Systematic Literature Review on Formalising Natural Language Requirements Using Large Language Models. *ArXiv preprint*. 2025. arXiv:2506.11874v1.

[3] COSLER, M., HAHN, C., MENDOZA, D., SCHMITT, F. and TRIPPEL, C. nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models. In: *Proceedings of the 35th International Conference on Computer Aided Verification (CAV)*. 2023.

[4] DANSO, P. K., HASAN, M. S., BALASUBRAMANIAN, N. and CHOWDHURY, O. Syntax Is Easy, Semantics Is Hard: Evaluating LLMs for LTL Translation. In: *Proceedings of the 2026 ACM Secure Development Conference (SecDev)*. 2026.

[5] ENGLISH, W., SIMON, D., JHA, S. K. and EWETZ, R. Grammar-Forced Translation of Natural Language to Temporal Logic using LLMs. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. 2025.

[6] FERRARI, A., GORI, G., ROSADINI, B., TROTTA, I., BACHERINI, S. et al. Detecting Requirements Defects with NLP Patterns: An Industrial Experience in the Railway Domain. *Empirical Software Engineering*. 2018.

[7] GAO, L., MADAAN, A., ZHOU, S., ALON, U., LIU, P. et al. PAL: Program-Aided Language Models. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2023.

[8] GENG, S., COOPER, H., MOSKAL, M., JENKINS, S., BERMAN, J. et al. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models. *ArXiv preprint*. 2025.

[9] GIANNAKOPOULOU, D., PRESSBURGER, T., MAVRIDOU, A., RHEIN, J., SCHUMANN, J. et al. Formal Requirements Elicitation with FRET. In: *Proceedings of the 26th International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ)*. 2020.

[10] HAHN, C., SCHMITT, F., TILLMAN, J. J., METZGER, N., SIBER, J. et al. Formal Specifications from Natural Language. In: *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. 2023.

[11] KHOT, T., TRIVEDI, H., FINLAYSON, M., FU, Y., RICHARDSON, K. et al. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. In: *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. 2023.

[12] MA, Z., WEN, C., SU, Z., LIANG, X., TIAN, C. et al. Bridging Natural Language and Formal Specification: Automated Translation of Software Requirements to LTL via Hierarchical Semantics Decomposition Using LLMs. *ArXiv preprint*. 2025.

[13] MAHBUB, T., DGHAYM, D., SHANKARNARAYANAN, A., SYED, T., SHAPSOUGH, S. et al. Can GPT-4 Aid in Detecting Ambiguities, Inconsistencies, and

Incompleteness in Requirements Analysis? A Comprehensive Case Study. *IEEE Access*. 2024.

[14] NOURI, A., CABRERO DANIEL, B., TÖRNER, F., SIVENCRONA, H. and BERGER, C. Engineering Safety Requirements for Autonomous Driving with Large Language Models. In: *Proceedings of the 32nd IEEE International Requirements Engineering Conference (RE)*. 2024.

[15] XU, Y., FENG, J. and MIAO, W. Learning from Failures: Translation of Natural Language Requirements into Linear Temporal Logic with Large Language Models. In: *Proceedings of the IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. 2024.

[16] YAN, Z., FANG, W., LI, M., LI, M., LIU, S. et al. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In: *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASPDAC)*. 2025.

1. Prompts

This appendix contains the system prompts used in the pipeline experiments. Each prompt consists of a role/goal statement, step-specific instructions, and (where applicable) the shared formalism context. The formalism context is identical across the entity extraction, constraint formalization, and chain-of-thought prompts; it is shown once below and omitted from subsequent prompts for brevity.

A.1 Shared Formalism Context

The following is an abbreviated version of the formalism context block included in the entity extraction, constraint formalization, and CoT prompts. The actual prompt lists each entity type with its permitted operations, each temporal construct with its parameter names, each predicate with its field structure, and each expression type with its arguments, along with a worked example of a complete formalization.

```
ENTITY TYPES: Signal (calculate), Storage (write, read), EventTrigger (fire),
Event (linked via modifiers), Channel (transmit, receive), State (set_state),
Value, Abstract, Set (insert, remove, clear)

TEMPORAL CONSTRUCTS: Always, Eventually, Initial, Causes, CausesWithin,
Sequence, Precedes, Excludes, Immediately

PREDICATES: Cmp, AllOf, AnyOf, Implies, Not, ForAll, Exists, Happening,
HasHappened, Operation

EXPRESSIONS: Constant, ArithOp, Now, Prev, Next, Val, ValBefore, EvtOccCount,
LastOcc, FirstOcc, MaxVal, MinVal, MkInterval, Start, End, Duration, Diff,
Addr, Size, Filter

IMPORTANT PATTERNS: EventTrigger + Event pairs: when an operation fires on
an entity, corresponding Event entities (linked via target and type modifiers)
become true. Time is ambient: expressions use Now() and temporal constructs
supply the time context.
```

Additionally, the prompts include a worked example showing the formalization of a scheduling deadline requirement (CausesWithin with Happening predicates).

A.2 Ambiguity Review

```
You are Ambiguity Reviewer. You are an expert in analyzing safety-critical
avionics requirements for ambiguity and potential misinterpretation.

Review requirements against the ambiguity checklist and flag genuine issues
without rewriting the requirement.

Checklist: dangling else / scope of action, ambiguity of reference, omissions
(causes without effects, effects without causes), ambiguous logical operators,
negation scope, vague verbs/adverbs/adjectives, built-in assumptions, ambiguous
precedence, implicit cases, temporal ambiguity, boundary ambiguity.

Do NOT rewrite or modify the requirement.
```

A.3 Flavour Extraction

```
You are a Time Model Classifier.

Discrete: step-by-step behavior, register operations, state transitions,
sequential actions without real-time durations.

Continuous: real-time durations (seconds, ms, ns), deadlines, continuous
physical quantities.
```

A.4 Entity Extraction

Uses the shared formalism context (Appendix A.1), followed by:

```
You are Entity Extractor.

Each entity has: name (unique identifier), type, modifiers (key-value pairs),
description. Every entity name must be unique.

IMPORTANT: For every EventTrigger, create a corresponding Event entity with
target and type modifiers. For operations that produce events, create
corresponding Event entities if the requirement references those occurrences.
```

A.5 Constraint Formalization (Pipeline)

Uses the shared formalism context, followed by a step-by-step decomposition algorithm:

```
You are Requirement Formalizer.
STEP 1 - IDENTIFY TOP-LEVEL TEMPORAL SCOPE: Always, Eventually,
Causes/CausesWithin, Sequence, Initial, or TraceAllOf.  Separate condition from
effect.
STEP 2 - DECOMPOSE CONDITION: Split into sub-predicates.  Event happening?
Value comparison?  Occurrence count?  Past event reference using LastOcc/Start?
STEP 3 - DECOMPOSE EFFECT: What value must the entity have?  Toggle (use
ValBefore)?  Just an event firing (use Happening)?
STEP 4 - NORMALIZE: Entity names must match exactly.  Operations must be
compatible with entity types.  Time references use Now(), Prev(Now()),
Start(LastOcc(...)), etc.
```

A.6 Chain-of-Thought (Single Call)

The CoT variant combines flavour extraction, entity extraction, and constraint formalization into a single prompt. It uses the same formalism context and adds a two-stage decomposition:

```
STAGE 1:  MACRO-STRUCTURE
Step 1:  Determine time model (Discrete/Continuous).  Step 2:  Identify
top-level temporal scope and separate condition from effect.
STAGE 2:  RECURSIVE DECOMPOSITION
Step 3:  Extract all entities with types and modifiers.  Step 4:  Decompose
condition into sub-predicates.  Step 5:  Decompose effect.  Step 6:  Normalize
entity references and operations.
Output the complete Requirement as JSON.
```